

Batch: A2 Roll No.: 16010324044

Experiment / assignment / tutorial No. 4

Grade: AA / AB / BB / BC / CC / CD / DD

Signature of the Staff In-charge with date

TITLE: Process Synchronization

Objectives: To Implement producer consumer problem using process Synchronization and implement Peterson's solution for 2-process communication.

OUTCOME: Student will be able to Compare different algorithms used for management and scheduling of processes.

1) **Peterson's Algorithm in Process Synchronization**

Peterson's Algorithm is a classic solution to the critical section problem in process synchronization. It ensures mutual exclusion meaning only one process can access the critical section at a time and avoids race conditions. The algorithm uses two shared variables to manage the turn-taking mechanism between two processes ensuring that both processes follow a fair order of execution. It's simple and effective for solving synchronization issues in two-process scenarios. In this article we will learn about Peterson's Algorithm, its working, and practical examples to help understand its use in process synchronization.

Algorithm for P _i process	Algorithm for P _j process
<pre> do{ flag[i] = true; turn = j; while (flag[j] && turn == j); //critical section flag[i] = false; //remainder section }while(true); </pre>	<pre> do{ flag[j] = true; turn = i; while (flag[i] && turn == i); //critical section flag[j] = false; //remainder section }while(true); </pre>

Peterson's Algorithm Explanation

Peterson's Algorithm is a mutual exclusion solution used to ensure that two processes do not enter into the critical sections at the same time. The algorithm uses two main components: a turn variable and a flag array.

- The turn variable is an integer that indicates whose turn it is to enter the critical section.
- The flag array contains Boolean values for each process, indicating whether a process wants to enter the critical section.

2) Producer consumer problem

DESCRIPTION

Producer consumer problem is a synchronization problem. There is a fixed size buffer where the producer produces items and that is consumed by a consumer process. One solution to the producer-consumer problem uses shared memory. To allow producer and consumer processes to run concurrently, there must be available a buffer of items that can be filled by the producer and emptied by the consumer. This buffer will reside in a region of memory that is shared by the producer and consumer processes. The producer and consumer must be synchronized, so that the consumer does not try to consume an item that has not yet been produced.

Code: (Process1)

```

#include <stdio.h>

#include <pthread.h>

#include <unistd.h>

```

```
int turn;

int flag[2] = {0,0};

void *process0(void *arg)
{
    for(int i=0;i<5;i++)
    {
        flag[0]=1;

        turn=1;

        while(flag[1] && turn==1);

        printf("Process 0 is in Critical Section\n");

        sleep(1);

        flag[0]=0;

        printf("Process 0 is in Remainder Section\n");
    }

    return NULL;
}

void *process1(void *arg)
{
    for(int i=0;i<5;i++)
    {
```

```
    flag[1]=1;

    turn=0;

    while(flag[0] && turn==0);

    printf("Process 1 is in Critical Section\n");

    sleep(1);

    flag[1]=0;

    printf("Process 1 is in Remainder Section\n");
}

return NULL;
}

int main()
{

    pthread_t t1,t2;

    pthread_create(&t1,NULL,process0,NULL);

    pthread_create(&t2,NULL,process1,NULL);

    pthread_join(t1,NULL);

    pthread_join(t2,NULL);

    return 0;
}
```

```
}
```

Output: (Process1)

```
Process 0 is in Critical Section
Process 0 is in Remainder Section
Process 1 is in Critical Section
Process 1 is in Remainder Section
Process 0 is in Critical Section
Process 0 is in Remainder Section
Process 1 is in Critical Section
Process 1 is in Remainder Section
Process 0 is in Critical Section
Process 0 is in Remainder Section
Process 1 is in Critical Section
Process 1 is in Remainder Section
Process 0 is in Critical Section
Process 0 is in Remainder Section
Process 1 is in Critical Section
Process 1 is in Remainder Section
Process 0 is in Critical Section
Process 0 is in Remainder Section
Process 1 is in Critical Section
Process 1 is in Remainder Section
```

Code: (Process1)

```
#include <stdio.h>

#include <pthread.h>

#include <semaphore.h>

#include <unistd.h>

#define BUFFER_SIZE 5
```

```
int buffer = 0;

sem_t empty, full, mutex;

void *producer(void *arg)
{
    for(int i=1;i<=5;i++)
    {
        sem_wait(&empty);

        sem_wait(&mutex);

        buffer++;

        printf("Produced Item %d\n", buffer);

        sem_post(&mutex);

        sem_post(&full);

        sleep(1);
    }

    return NULL;
}

void *consumer(void *arg)
{
    for(int i=1;i<=5;i++)
```

```
{  
  
    sem_wait(&full);  
  
    sem_wait(&mutex);  
  
    printf("Consumed Item %d\n", buffer);  
  
    buffer--;  
  
    sem_post(&mutex);  
  
    sem_post(&empty);  
  
    sleep(2);  
}  
  
return NULL;  
}  
  
int main()  
{  
  
    pthread_t p,c;  
  
    sem_init(&empty,0,BUFFER_SIZE);  
  
    sem_init(&full,0,0);  
  
    sem_init(&mutex,0,1);  
  
    pthread_create(&p,NULL,producer,NULL);  
  
    pthread_create(&c,NULL,consumer,NULL);  

```

```
pthread_join(p, NULL);  
  
pthread_join(c, NULL);  
  
sem_destroy(&empty);  
sem_destroy(&full);  
sem_destroy(&mutex);  
  
return 0;  
}
```

Output: (Process2)

```
Produced Item 1  
Consumed Item 1  
Produced Item 1  
Consumed Item 1  
Produced Item 1  
Produced Item 2  
Consumed Item 2  
Produced Item 2  
Consumed Item 2  
Consumed Item 1
```

Conclusion:

The experiment demonstrated the importance of process synchronization in operating systems. Peterson's Algorithm provides software-based mutual exclusion for two processes using shared variables, while the Producer-Consumer problem uses semaphores to synchronize access to a shared buffer. Both implementations prevent race conditions and ensure correct execution of concurrent processes.

Signature of faculty in-charge

Department of Electronics and Telecommunication Engineering